

AttestFlow — Cross-Chain Verified Payment Engine

AttestFlow: Cross-Chain Verified Payment Engine

BUIDL CTC 2026 Fall — Track: AI *An autonomous AI agent that turns real cross-chain payments into cryptographically verified, automatically settled outcomes — powered by the Attestcoin Protocol.*

1. Problem

Cross-chain “payment acceptance” today relies on either centralized bridges (custody risk, opaque off-chain operators) or oracle networks that deliver *data about* payments but rarely carry *guarantees about* them. For a freelancer waiting on an invoice, or a business reconciling cross-border receivables, three things are missing:

1. **Trustless proof** that a specific payment really happened and succeeded on the source chain.
2. **Autonomous execution** the moment that proof exists — no human reconciling invoices.
3. **A closed loop**: the outcome must travel back to the other chain as a verified message, not a screenshot.

Meanwhile, LLM-based agents excel at interpreting intent (“watch my incoming invoices, release funds at 10%”) but hallucinate when asked to trust unverified chain data.

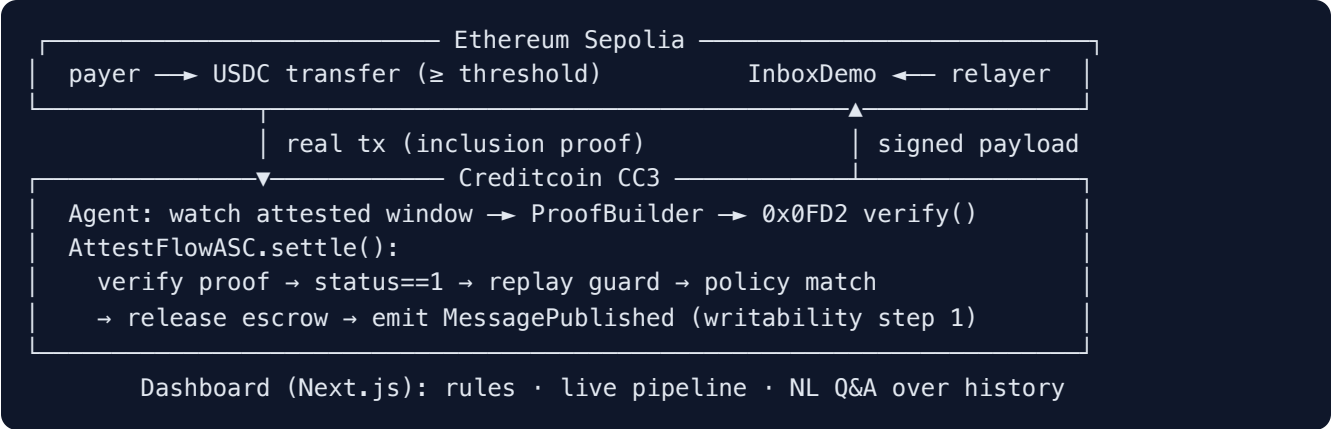
2. Solution

AttestFlow is an agent-native payment engine that separates **intent** (natural language), **truth** (Attestcoin Protocol cryptographic proofs), and **execution** (a deterministic smart-contract state machine):

- **Intent layer** — a natural-language rule (“当我在 Sepolia 收到 ≥ 100 USDC 时，按 10% 释放”) is compiled into a structured policy by a deterministic parser (LLM optional, disclosed). Determinism here is a feature: money rules must be reproducible.
- **Truth layer** — the Attestcoin Protocol’s readability proves, *on Creditcoin*, that a specific Sepolia transaction is included in a confirmed source-chain block. The proof is generated by the official hosted prover and verified by the native `0x0FD2` precompile at EVM speed.
- **Execution layer** — `AttestFlowASC` (an Attestcoin Smart Contract) re-checks what the protocol does not cover — receipt success, replay, policy match — then releases escrowed CTC and emits a writability message. A relayer delivers it to Sepolia, where an validating inbox executes it.

The agent is the *muscle* (watching, proving, relaying), the ASC is the *spine* (trust), and the protocol is the *nervous system* connecting two chains.

3. Architecture



4. Attestcoin Protocol integration depth

Protocol surface	Usage
ChainInfo precompile	Runtime discovery of supported source chains (chainKey), attested heights
Proof generation	Official hosted prover (ProofBuilder), Merkle + continuity proofs
BlockProver 0x0FD2	Synchronous in-ASC verification (verify()) of every settled payment
Encoding v1	On-chain decoding of attested transaction bytes, incl. receipt status
Writability semantics	Four-step publish→sign→deliver→validate loop (adapter until official Outbox/Inbox ship)

Engineering findings contributed back (detailed in docs/integration.md):

- Contract-context verifyAndEmit() reverts on CC3 testnet; verify() is the correct in-ASC primitive today.
- Real Sepolia USDC traffic is dominated by transferFrom (selector 0x23b872dd); policies must match calldata, not just Transfer events.
- Scanning the attested window (prover REST) eliminates attestation-latency stalls while remaining fully proof-backed.

5. Security model

1. **Cryptographic**: every settlement requires a valid Merkle + continuity proof against attestor-committed checkpoints. No agent decision can mint a settlement without it.
2. **Beyond inclusion**: the protocol proves inclusion, not success. The ASC decodes the attested receipt and rejects `status != 1` — a dedicated test covers this adversarial path.
3. **Replay**: `keccak(chainKey, height, txIndex)` is single-use on-chain; the agent additionally dedupes per tx hash across restarts.
4. **Least trust in the agent**: the relayer/operator cannot forge settlements (needs a valid proof), cannot double-settle (on-chain guard), cannot redirect funds (beneficiary is policy-fixed). Its only power is *when* to submit.
5. **Deterministic money rules**: policy compilation is deterministic by default; optional LLM parsing is disclosed and never used for settlement decisions.

6. Results (public testnets, reproducible)

- 6+ autonomous on-chain settlements driven by real third-party USDC payments.
- Example loop: source payment `0x39091951...` → settlement `0x9c8caf6b...` (CC3, 190–195k gas) → verified delivery `0xd6abf721...` (Sepolia).
- 14/14 contract tests including the security-critical rejection paths.
- End-to-end agent run (NL rule → settle → deliver) completes unattended in ~40 s once a matching payment exists in the attested window.

7. Roadmap

1. **Official writability swap** — replace the relayer adapter with Outbox/Inbox when they deploy on testnet (payload shape already aligned).
2. **Multi-source policies** — same engine, more chainKeys via ChainInfo registry.
3. **Quorum inbox** — attestor-set signature verification replacing the single authorized relayer.
4. **Invoice NFT receipts** — settlement mints a transferable proof-of-payment asset.
5. **LLM risk copilot** — anomaly commentary over settlement history (display-only; never on the money path).

8. Conclusion

AttestFlow demonstrates that the Attestcoin Protocol is sufficient — today, on public testnets — to power *autonomous, proof-gated cross-chain payments* with an AI agent in the driver's seat and cryptography in the guardrails. Every claim in this paper is backed by a transaction hash a judge can click.